



UNIVERSIDAD PRIVADA DEL VALLE
FACULTAD DE INFORMÁTICA Y ELECTRÓNICA
CARRERA DE LICENCIATURA EN INGENIERÍA DE
SISTEMAS INFORMÁTICOS

PROGRAMACIÓN WEB II

SISTEMA DE REGULACIÓN DE INGRESOS Y EGRESOS FAMILIARES

Paralelo: “B”

Estudiantes: José Martín Leño Mercado

Melani Quintela Aguilar

José Alejandro Salazar Quiroga

Docente: Ing. Henry Luis Miranda Ordoñez

La Paz – Bolivia

Gestión I – 2026

WALLET FAMILY

Sistema de Gestión Financiera Familiar

TABLA DE CONTENIDO

TABLA DE CONTENIDO	2
INTRODUCCIÓN	5
Objetivo del Sistema.....	5
Alcance	5
1. REQUERIMIENTOS TÉCNICOS	6
1.1 Requisitos del Servidor (Backend).....	6
1.2 Requisitos del Cliente (Frontend)	6
1.3 Dependencias del Proyecto (composer.json).....	6
2. HERRAMIENTAS UTILIZADAS PARA EL DESARROLLO	7
2.1 PHP 8.0	7
2.2 MySQL 8.0	7
2.3 Apache (XAMPP)	7
2.4 Laravel 9.....	7
2.5 Bootstrap 5	8
2.6 JavaScript (Vanilla ES6+)	8
3. INSTALACIÓN DEL WEB SERVER Y ENTORNO	9
3.1 Instalación de XAMPP	9
3.2 Preparación del Proyecto Laravel (Backend).....	9
3.3 Instalación del Frontend	9
4. CONFIGURACIÓN DEL APLICATIVO	10
4.1 Configurar el Archivo .env	10
4.2 Configurar la Base de Datos.....	10
4.3 Configurar CORS	10
4.4 Activar el Scheduler de Laravel (Recordatorios SMS)	11
5. ENDPOINTS DE LA API REST	12
5.1 Autenticación	12
5.2 Dashboard	13
5.3 Ingresos	13
5.4 Gastos	14
5.5 Categorías	16

5.6 Pagos Programados	17
5.7 Perfil y Usuarios	19
5.8 Panel de Control Familiar	19
5.9 Suscripciones	20
5.10 Auditoría y Reportes	21
6. CASOS DE USO	22
6.1 Actores del Sistema.....	22
6.2 Descripción de Casos de Uso Principales	22
CU-01: Registro e Incorporación a Familia.....	22
CU-02: Registro de Ingreso	22
CU-03: Registro de Gasto con Validación de Saldo	23
CU-04: Gestión de Pago Programado.....	23
CU-05: Activación de Plan Premium	23
CU-06: Generación de Reportes Financieros.....	24
7. MÓDULO DE ADMINISTRACIÓN.....	25
7.1 Panel de Control Familiar (panel-control.html)	25
7.2 Gestión de Suscripciones (admin_suscripciones.html)	25
7.3 Gestión de Categorías (categorias.html).....	25
7.4 Auditoría de Seguridad (auditoria.html)	25
8. MODELO ENTIDAD RELACIÓN.....	26
8.1 Relaciones Principales	26
9. DICCIONARIO DE DATOS	28
9.1 Tabla: users.....	28
9.2 Tabla: families	28
9.3 Tabla: incomes (Ingresos)	29
9.4 Tabla: expenses (Gastos).....	29
9.5 Tabla: scheduled_payments (Pagos Programados)	29
9.6 Tabla: categories	30
9.7 Tabla: subscriptions.....	30
9.8 Tabla: plans.....	31
10. PROTOTIPOS DE PANTALLAS DEL APLICATIVO.....	32
10.1 Inventario de Pantallas	32
10.2 Estructura de Navegación Común	33
10.3 Descripción Detallada de Pantallas Clave	33
Dashboard (dashboard.html).....	33
Registro (registro.html).....	33

Suscripción (suscripcion.html).....	33
11. ARQUITECTURA DEL SISTEMA Y SEGURIDAD	34
11.1 Arquitectura General.....	34
11.2 Mecanismos de Seguridad Implementados	34

INTRODUCCIÓN

Wallet Family es una aplicación web orientada a la gestión financiera colaborativa dentro del núcleo familiar. Permite a los miembros de una familia registrar ingresos, controlar gastos, programar pagos recurrentes y generar reportes financieros, todo bajo un esquema multiusuario con aislamiento de datos por familia.

El presente manual técnico documenta la arquitectura completa del sistema, los requisitos de infraestructura, la instalación del entorno de desarrollo, la configuración de la aplicación, los endpoints de la API REST, los modelos de datos y las pantallas del módulo administrativo. Está dirigido a desarrolladores, técnicos de sistemas y docentes evaluadores que necesiten comprender o desplegar el proyecto.

Objetivo del Sistema

Proporcionar a las familias una herramienta digital accesible para:

- **Registrar y categorizar** ingresos y egresos familiares o personales.
- **Programar pagos** con recordatorio automático (Twilio SMS).
- **Controlar el saldo** disponible en tiempo real con validación de saldo negativo.
- **Gestionar suscripciones** en planes Trial y Premium con verificación manual de pago.
- **Auditar acciones** mediante procedimientos almacenados de MySQL.
- **Generar reportes** financieros mensuales, anuales y por categoría/miembro.

Alcance

El sistema cubre dos grandes módulos:

- **Backend (API REST):** Construido con Laravel 9, expone endpoints JSON consumidos por el frontend. Gestiona la lógica de negocio, seguridad y base de datos.
- **Frontend (SPA estático):** Páginas HTML5 que consumen la API mediante fetch(). Usan Bootstrap 5 para el diseño responsivo y JavaScript vanilla para la interactividad.

1. REQUERIMIENTOS TÉCNICOS

1.1 Requisitos del Servidor (Backend)

Componente	Versión Mínima	Recomendada
PHP	8.0	8.1+
Laravel	9.x	9.52+
MySQL	8.0	8.0.30+
Apache	2.4 / 1.18	Cualquiera reciente
Composer	2.x	2.5+

1.2 Requisitos del Cliente (Frontend)

Requisito	Detalle
Navegador	Chrome 90+, Firefox 88+, Edge 90+, Safari 14+
JavaScript	ES6+ habilitado (fetch, async/await, localStorage)
Conexión	Acceso a la URL base de la API (misma red o Internet)
Resolución	Mínimo 360px de ancho (diseño mobile-first con Bootstrap 5)

1.3 Dependencias del Proyecto (composer.json)

Las siguientes librerías PHP son instaladas automáticamente con Composer:

Paquete	Versión	Propósito
laravel/framework	^9.0	Framework MVC principal
laravel/sanctum	^2.14	Autenticación stateless con Bearer Token
fruitcake/laravel-cors	^2.0.5	Manejo de encabezados CORS para el frontend
twilio/sdk	^8.11	Envío de SMS para recordatorios de pagos
guzzlehttp/guzzle	^7.2	Cliente HTTP para integraciones externas

2. HERRAMIENTAS UTILIZADAS PARA EL DESARROLLO

2.1 PHP 8.0

PHP (Hypertext Preprocessor) es el lenguaje de programación del lado del servidor sobre el que está construido Laravel. La versión 8 introduce mejoras de rendimiento (JIT Compiler), tipos de unión, atributos nativos y constructores de propiedades promovidas, que Laravel 9 aprovecha para reducir el código repetitivo (boilerplate).

□ *Wallet Family utiliza PHP para toda la lógica de negocio: validación de datos, operaciones en base de datos, generación de tokens y comunicación con la API de Twilio.*

2.2 MySQL 8.0

MySQL es el Sistema de Gestión de Bases de Datos Relacionales (RDBMS) elegido para este proyecto. Almacena todas las entidades del sistema (familias, usuarios, ingresos, gastos, etc.) y contiene los Procedimientos Almacenados (Stored Procedures) que ejecutan la lógica de auditoría y generación de reportes directamente en el motor de base de datos, reduciendo la carga en el servidor de aplicaciones.

□ *Se utilizan Procedimientos Almacenados para: `sp_auditoria_usuario`, `sp_auditoria_familia`, `sp_historial_reportes`, `sp_reporte_mensual`, `sp_reporte_anual`, `sp_top_categorias`, `sp_reporte_por_miembro`, `sp_reporte_pagos_programados`.*

2.3 Apache (XAMPP)

Apache HTTP Server es el servidor web utilizado en el entorno de desarrollo local a través del stack XAMPP (Cross-Platform, Apache, MySQL, PHP, Perl). Se configura para procesar las solicitudes hacia la aplicación Laravel usando el archivo `.htaccess` en la carpeta `/public`, que dirige todo el tráfico al punto de entrada `index.php`.

□ *En producción puede usarse Nginx con configuración similar. El archivo `.env` controla la URL base del proyecto.*

2.4 Laravel 9

Laravel es el framework PHP MVC (Modelo-Vista-Controlador) que estructura el backend del sistema. En Wallet Family actúa exclusivamente como una API REST sin vistas Blade, retornando únicamente respuestas JSON. Las características principales que se utilizan son:

- **Eloquent ORM:** Mapeo objeto-relacional para interactuar con MySQL sin escribir SQL crudo.
- **Sanctum:** Autenticación basada en tokens Bearer para proteger los endpoints.
- **Rutas API:** Definidas en `routes/api.php` con prefijo `/api` automático.
- **Policies:** Control de autorización granular (`ExpensePolicy`, `IncomePolicy`, `ScheduledPaymentPolicy`).

- **Middlewares:** Autenticación, CORS y protección CSRF gestionados de forma centralizada.
- **Migrations:** Control de versiones del esquema de base de datos.
- **Console Commands:** Tarea programada SendPaymentReminders para recordatorios automáticos por SMS.

2.5 Bootstrap 5

Bootstrap 5 es el framework CSS utilizado en todas las páginas HTML del frontend para garantizar un diseño responsivo y consistente. Se consume directamente desde CDN, por lo que no requiere proceso de compilación. Se usa para el layout de sidebar/contenido, cards, modals, tablas, formularios, badges y el sistema de grid responsivo.

2.6 JavaScript (Vanilla ES6+)

Todas las páginas HTML utilizan JavaScript nativo (sin frameworks como React o Vue) para consumir la API REST mediante la función `fetch()`. Los patrones usados incluyen `async/await` para solicitudes asíncronas, `localStorage` para persistir el token de sesión, manipulación del DOM para renderizar datos dinámicamente y manejo de eventos para los formularios de CRUD.

3. INSTALACIÓN DEL WEB SERVER Y ENTORNO

3.1 Instalación de XAMPP

XAMPP integra Apache, MySQL y PHP en un único paquete de instalación multiplataforma. Es la forma más sencilla de levantar el entorno de desarrollo local para Wallet Family.

1. Descargar XAMPP desde: <https://www.apachefriends.org/es/index.html> (versión con PHP 8.0 o superior).
2. Ejecutar el instalador como Administrador. Seleccionar los componentes: Apache, MySQL, PHP, phpMyAdmin.
3. Finalizar la instalación. La ruta por defecto en Windows es C:\xampp.
4. Abrir el Panel de Control de XAMPP y hacer clic en Start en los módulos Apache y MySQL.
5. Verificar en el navegador: <http://localhost> debe mostrar la página de bienvenida de XAMPP.

3.2 Preparación del Proyecto Laravel (Backend)

6. Copiar o descomprimir la carpeta walletFamily dentro de C:\xampp\htdocs\.
7. Abrir una terminal o CMD en la ruta C:\xampp\htdocs\walletFamily.
8. Instalar las dependencias PHP con el siguiente comando:

```
composer install
```

9. Crear el archivo de entorno copiando el ejemplo:

```
cp .env.example .env
```

10. Generar la clave de cifrado de la aplicación:

```
php artisan key:generate
```

11. Ejecutar el enlace del almacenamiento para imágenes y comprobantes:

```
php artisan storage:link
```

3.3 Instalación del Frontend

El frontend no requiere proceso de compilación ni servidor especial. Los archivos HTML se sirven estáticamente:

12. Copiar la carpeta walletFamily-Front dentro de C:\xampp\htdocs\.
13. Acceder a las páginas desde el navegador: <http://localhost/walletFamily-Front/index.html>

□ Los archivos HTML se comunican con el backend a través de `fetch()` apuntando a la URL base de la API definida en cada script JavaScript. Verificar que la constante `API_URL` en cada página apunte a <http://localhost/walletFamily/public/api>.

4. CONFIGURACIÓN DEL APLICATIVO

4.1 Configurar el Archivo .env

El archivo .env ubicado en la raíz del proyecto Laravel concentra toda la configuración sensible del sistema. Los parámetros críticos a editar son:

```
APP_NAME=WalletFamily
APP_ENV=local
APP_KEY=base64:... ← Se genera con php artisan key:generate
APP_DEBUG=true
APP_URL=http://localhost/walletFamily/public
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=wallet_family
DB_USERNAME=root
DB_PASSWORD= ← Vacío en XAMPP por defecto
```

```
TWILIO_SID=ACxxxxxxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=your_auth_token
TWILIO_PHONE=+1XXXXXXXXXX
```

4.2 Configurar la Base de Datos

14. Abrir phpMyAdmin en: <http://localhost/phpmyadmin>
15. Crear una nueva base de datos con nombre: wallet_family y cotejamiento utf8mb4_unicode_ci.
16. En la terminal del proyecto, ejecutar las migraciones para crear todas las tablas:

```
php artisan migrate
```

17. Ejecutar los seeders para poblar los datos iniciales (roles, planes, usuario administrador):

```
php artisan db:seed
```

□ *El usuario Administrador (Super Admin) se crea con ID=1. Las funcionalidades de administración global (verificar pagos, ver suscripciones) están restringidas a este usuario.*

4.3 Configurar CORS

El archivo config/cors.php controla qué orígenes pueden consumir la API. Para desarrollo local con el frontend estático, la configuración permite todos los orígenes:

```
'allowed_origins' => ['*'],
'allowed_methods' => ['*'],
'allowed_headers' => ['*'],
```

Para producción, reemplazar el * con la URL exacta del dominio del frontend.

4.4 Activar el Scheduler de Laravel (Recordatorios SMS)

El sistema incluye el comando `SendPaymentReminders` que se ejecuta diariamente para enviar SMS a usuarios con pagos próximos a vencer. Para activarlo en Windows:

18. Abrir el Programador de Tareas de Windows.

19. Crear una tarea que se ejecute cada minuto con el siguiente comando:

```
| php C:\xampp\htdocs\walletFamily\artisan schedule:run
```

En Linux/Mac, agregar al crontab:

```
| * * * * * cd /ruta/del/proyecto && php artisan schedule:run >> /dev/null 2>&1
```

5. ENDPOINTS DE LA API REST

Todos los endpoints tienen el prefijo base: /api. Los que requieren autenticación deben enviar el encabezado:

```
Authorization: Bearer {token}
```

5.1 Autenticación

Método	Endpoint	Descripción	Auth
POST	/api/login	Iniciar sesión y obtener token Bearer	No
POST	/api/registro	Registrar usuario (crear o unirse a familia)	No
POST	/api/logout	Cerrar sesión y revocar token actual	Sí
GET	/api/user	Obtener datos del usuario autenticado	Sí

Ejemplo de respuesta exitosa de /api/login:

```
{ "message": "Login exitoso",  
  "access_token": "1|abc123...",  
  "token_type": "Bearer",  
  "user": { "id": 1, "name": "Juan", "email": "..." } }
```

POST ⌵ http://127.0.0.1:8000/api/login Send

QueryHeaders 2AuthBody 1TestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

Form Fields

☐ FilesImport

☒ emaillia@gmail.com

☒ password654321

☐ field namevalue

Learn more about how to set a custom [content-type](#) for a field.

Status: 200 OKSize: 468 BytesTime: 241 ms

ResponseHeaders 9CookiesResultsDocs</>≡

```
1 {  
2   "message": "Login exitoso",  
3   "access_token":  
4     "1|lzuKzWejdjcUCFziRRK4xfBkmYtOEN6nIrtfJHh",  
5   "token_type": "Bearer",  
6   "user": {  
7     "id": 12,  
8     "name": "lia",  
9     "email": "lia@gmail.com",  
10    "email_verified_at": null,  
11    "created_at": "2026-06-09T01:21:33.000000Z",  
12    "updated_at": "2026-06-09T01:21:33.000000Z",  
13    "last_name": "me",  
14    "second_last_name": "me",  
15    "birth_date": "2000-10-02T00:00:00.000000Z",  
16    "phone": "12345678",  
17    "profile_photo": null,  
18    "family_member": "mother",  
19    "status": "active",  
20    "family_id": 12,  
21    "role_id": 1  
22  }  
}
```

5.2 Dashboard

Método	Endpoint	Descripción	Auth
GET	/api/dashboard	Resumen financiero: totales y últimos movimientos. Parámetros opcionales: ?vista=personal familiar&limite=5 20 40	Sí

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: GET
 - URL: http://127.0.0.1:8000/api/dashboard
 - Auth: Bearer Token
 - Token: 1||zuKzWejdjcUCFZiRRK4xfaBKmYtOEN6nlrtfJHh
- Response:**
 - Status: 200 OK
 - Size: 228 Bytes
 - Time: 176 ms
 - JSON Body:

```

1 {
2   "message": "Datos del dashboard recuperados exitosamente",
3   "data": {
4     "configuracion": {
5       "vista_actual": "familiar",
6       "limite_movimientos": 5
7     },
8     "resumen": {
9       "total_ingresos": 0,
10      "total_gastos": 0,
11      "saldo_disponible": 0
12    },
13    "ultimos_movimientos": []
14  }
15 }
```

5.3 Ingresos

Método	Endpoint	Descripción	Auth
GET	/api/ingresos	Listar ingresos del mes. Parámetro: ?month=YYYY-MM	Sí
POST	/api/ingresos	Registrar nuevo ingreso	Sí
PUT	/api/ingresos/{income}	Actualizar ingreso existente	Sí
DELETE	/api/ingresos/{income}	Eliminar ingreso	Sí

POST http://127.0.0.1:8000/api/ingresos Send

Query Headers 2 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "description": "Sueldo de la quincena",
3   "amount": 3500.50,
4   "date": "2026-06-09",
5   "category_id": 1
6 }
```

Status: 201 Created Size: 907 Bytes Time: 170 ms

Response Headers 9 Cookies Results Docs </> ≡

```
1 {
2   "message": "Ingreso registrado correctamente",
3   "data": {
4     "description": "Sueldo de la quincena",
5     "amount": "3500.50",
6     "income_date": "2026-06-09T00:00:00.000000Z",
7     "user_id": 12,
8     "category_id": 1,
9     "updated_at": "2026-06-09T13:21:00.000000Z",
10    "created_at": "2026-06-09T13:21:00.000000Z",
11    "id": 31,
12    "category": {
13      "id": 1,
14      "name": "Supermercado 9228",
15      "type": "scheduled_payment",
16      "description": "Et numquam labore fugit quae at quia consequuntur laborum.",
17      "status": "inactive",
18      "family_id": 6,
19      "created_at": "2026-06-09T01:09:54.000000Z",
20      "updated_at": "2026-06-09T01:09:54.000000Z"
21    },
22    "user": {
23      "id": 12,
24      "name": "lia",
25      "email": "lia@gmail.com",
26      "email_verified_at": null,
27      "created_at": "2026-06-09T01:21:33.000000Z",
28      "updated_at": "2026-06-09T01:21:33.000000Z",
29      "last_name": "me",
30      "second_last_name": "me",
31      "birth_date": "2000-10-02T00:00:00.000000Z",
32      "phone": "12345678",
33      "password": "12345678"
34    }
35  }
36 }
```

5.4 Gastos

Método	Endpoint	Descripción	Auth
GET	/api/gastos	Listar gastos del mes. Parámetro: ?month=YYYY-MM	Sí
POST	/api/gastos	Registrar nuevo gasto (valida saldo disponible)	Sí
PUT	/api/gastos/{expense}	Actualizar gasto existente	Sí
DELETE	/api/gastos/{expense}	Eliminar gasto	Sí

POST ⌵ http://127.0.0.1:8000/api/gastos Send

Query Headers ² Auth ¹ Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "description": "Cena familia",
3   "amount": 150.50,
4   "date": "2026-06-08",
5   "category_id": 1
6 }
```

GET ⌵ http://127.0.0.1:8000/api/gastos Send

Query Headers ² Auth ¹ Body Tests Pre Run

None Basic Bearer OAuth 2 NTLM AWS

Bearer Token

1|zuKzWejdjcUCFZIRRK4xfaBKmYtOEN6nlrtfJHh

Token Prefix Bearer

Status: 201 Created Size: 896 Bytes Time: 196 ms

Response Headers ⁹ Cookies Results Docs </> ≡

```
1 {
2   "message": "Gasto registrado correctamente",
3   "data": {
4     "description": "Cena familia",
5     "amount": "150.50",
6     "expense_date": "2026-06-08T00:00:00.000000Z",
7     "user_id": 12,
8     "category_id": 1,
9     "updated_at": "2026-06-09T13:06:33.000000Z",
10    "created_at": "2026-06-09T13:06:33.000000Z",
11    "id": 51,
12    "category": {
13      "id": 1,
14      "name": "Supermercado 9228",
15      "type": "scheduled_payment",
16      "description": "Et numquam labore fugit quae at quia consequuntur laborum.",
17      "status": "inactive",
18      "family_id": 6,
19      "created_at": "2026-06-09T01:09:54.000000Z",
20      "updated_at": "2026-06-09T01:09:54.000000Z"
21    },
22    "user": {
23      "id": 12,
24      "name": "lia",
25      "email": "lia@gmail.com",
26      "email_verified_at": null,
27      "created_at": "2026-06-09T01:21:33.000000Z",
28      "updated_at": "2026-06-09T01:21:33.000000Z",
29      "last_name": "me",
30      "second_last_name": "me",
31      "birth_date": "2000-10-02T00:00:00.000000Z",
32      "phone": "12345678",
```

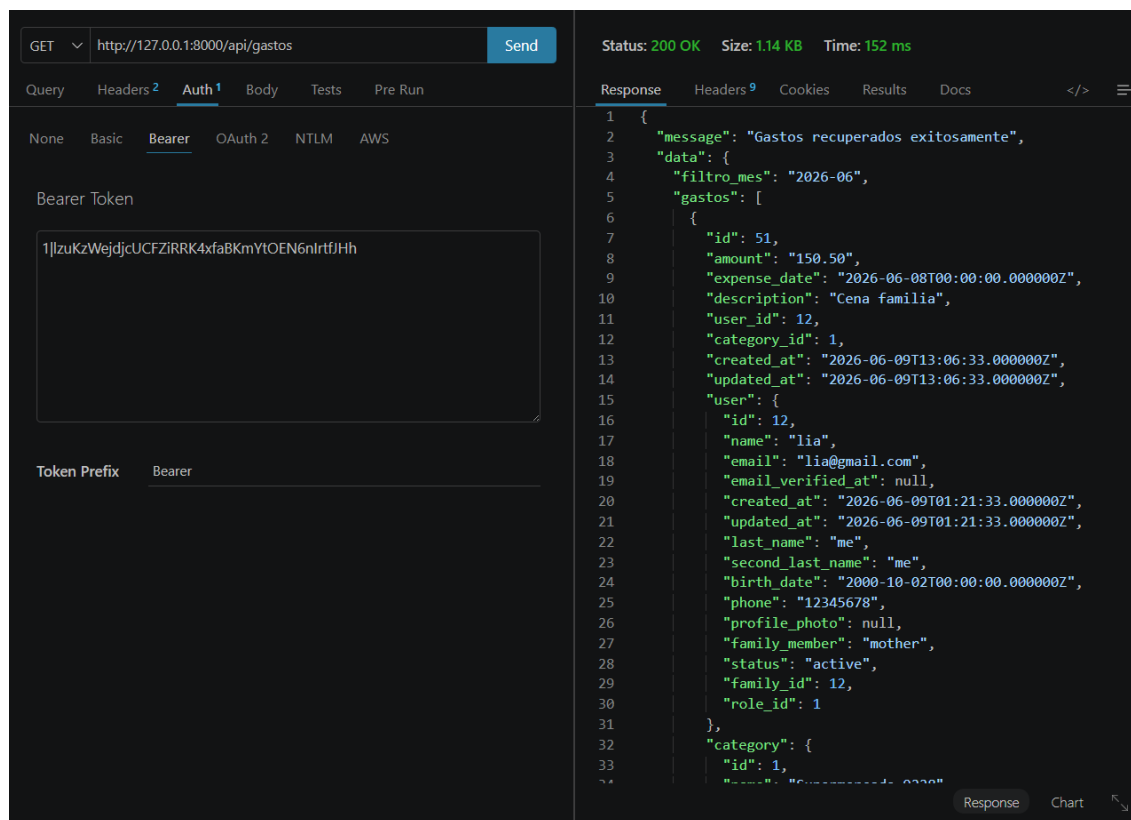
Copy

Status: 200 OK Size: 1.14 KB Time: 152 ms

Response Headers ⁹ Cookies Results Docs </> ≡

```
1 {
2   "message": "Gastos recuperados exitosamente",
3   "data": {
4     "filtro_mes": "2026-06",
5     "gastos": [
6       {
7         "id": 51,
8         "amount": "150.50",
9         "expense_date": "2026-06-08T00:00:00.000000Z",
10        "description": "Cena familia",
11        "user_id": 12,
12        "category_id": 1,
13        "created_at": "2026-06-09T13:06:33.000000Z",
14        "updated_at": "2026-06-09T13:06:33.000000Z",
15        "user": {
16          "id": 12,
17          "name": "lia",
18          "email": "lia@gmail.com",
19          "email_verified_at": null,
20          "created_at": "2026-06-09T01:21:33.000000Z",
21          "updated_at": "2026-06-09T01:21:33.000000Z",
22          "last_name": "me",
23          "second_last_name": "me",
24          "birth_date": "2000-10-02T00:00:00.000000Z",
25          "phone": "12345678",
26          "profile_photo": null,
27          "family_member": "mother",
28          "status": "active",
29          "family_id": 12,
30          "role_id": 1
31        },
32        "category": {
33          "id": 1,
34          "name": "Supermercado 9228",
```

Response Chart ⌵



5.5 Categorías

Método	Endpoint	Descripción	Auth
GET	<code>/api/categorias</code>	Listar categorías activas de la familia	Sí
POST	<code>/api/categorias</code>	Crear nueva categoría (tipo: <code>income expense scheduled_payment</code>)	Sí
PUT	<code>/api/categorias/{category}</code>	Editar nombre o descripción de categoría	Sí
PUT	<code>/api/categorias/{category}/toggle</code>	Activar / desactivar categoría	Sí

Request 1:

Method: POST
URL: http://127.0.0.1:8000/api/categorias
Status: 201 Created, Size: 267 Bytes, Time: 189 ms

Body (JSON):

```
{
  "name": "Supermercado",
  "type": "expense",
  "description": "Compras semanales de comida"
}
```

Response 1:

```
{
  "message": "Categoría creada exitosamente",
  "data": {
    "name": "Supermercado",
    "type": "expense",
    "description": "Compras semanales de comida",
    "status": "active",
    "family_id": 12,
    "updated_at": "2026-06-09T12:52:27.000000Z",
    "created_at": "2026-06-09T12:52:27.000000Z",
    "id": 21
  }
}
```

Request 2:

Method: GET
URL: http://127.0.0.1:8000/api/categorias
Status: 200 OK, Size: 277 Bytes, Time: 158 ms

Body (JSON):

```
[
  {
    "id": 21,
    "name": "Supermercado",
    "type": "expense",
    "description": "Compras semanales de comida",
    "status": "active",
    "family_id": 12,
    "created_at": "2026-06-09T12:52:27.000000Z",
    "updated_at": "2026-06-09T12:52:27.000000Z"
  }
]
```

5.6 Pagos Programados

Método	Endpoint	Descripción	Auth
GET	/api/pagos	Listar pagos programados. Filtros: ?month=YYYY-MM&status=pending paid overdue	Sí
POST	/api/pagos	Crear nuevo pago programado	Sí
PUT	/api/pagos/{scheduledPayment}	Actualizar estado del pago (pending paid overdue)	Sí
DELETE	/api/pagos/{scheduledPayment}	Eliminar pago programado	Sí

GET ⌵ http://127.0.0.1:8000/api/pagos?status=pending Send

Query Headers **Auth**¹ Body Tests Pre Run

None Basic Bearer OAuth 2 NTLM AWS

Bearer Token

1||zuKzWejdjcUCFZiRRK4xfaBKmYtOEN6nlrtfJHh

Token Prefix Bearer

Status: 200 OK Size: 104 Bytes Time: 162 ms

Response Headers⁹ Cookies Results Docs </> ≡

```
1 {
2   "message": "Pagos programados recuperados exitosamente",
3   "data": {
4     "pagos": [],
5     "categorias_disponibles": []
6   }
7 }
```

POST ⌵ http://127.0.0.1:8000/api/pagos Send

Query Headers **Auth**¹ **Body**¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "name": "Pago de Internet mensual",
3   "amount": 250.50,
4   "deadline_date": "2026-06-15",
5   "category_id": 1
6 }
```

Status: 201 Created Size: 931 Bytes Time: 138 ms

Response Headers⁹ Cookies Results Docs </> ≡

```
1 {
2   "message": "Pago programado registrado correctamente",
3   "data": {
4     "name": "Pago de Internet mensual",
5     "amount": "250.50",
6     "deadline_date": "2026-06-15T00:00:00.000000Z",
7     "status": "pending",
8     "user_id": 12,
9     "category_id": 1,
10    "updated_at": "2026-06-09T13:35:20.000000Z",
11    "created_at": "2026-06-09T13:35:20.000000Z",
12    "id": 16,
13    "user": {
14      "id": 12,
15      "name": "lia",
16      "email": "lia@gmail.com",
17      "email_verified_at": null,
18      "created_at": "2026-06-09T01:21:33.000000Z",
19      "updated_at": "2026-06-09T01:21:33.000000Z",
20      "last_name": "me",
21      "second_last_name": "me",
22      "birth_date": "2000-10-02T00:00:00.000000Z",
23      "phone": "12345678",
24      "profile_photo": null,
25      "family_member": "mother",
26      "status": "active",
27      "family_id": 12,
28      "role_id": 1
29    },
30    "category": {
31      "id": 1,
32      "name": "Supermercado 9228",
33      "type": "scheduled_payment",
34      "description": "Et cumsum isbore fugit quis et quis
```

5.7 Perfil y Usuarios

Método	Endpoint	Descripción	Auth
PUT	/api/perfil/datos/{user}	Actualizar datos personales del usuario autenticado	Sí
PUT	/api/perfil/foto/{user}	Actualizar foto de perfil (multipart/form-data, max 2MB)	Sí
PUT	/api/usuarios/{user}	Editar datos de un miembro de la familia	Sí
DELETE	/api/usuarios/{user}	Expulsar miembro de la familia	Sí

POST

Query Headers ² Auth ¹ **Body ¹** Tests Pre Run

JSON XML Text **Form** Form-encode GraphQL Binary

Form Fields ☒ Files

☒ _method PUT

☐ field name value

Files

☒ profile_photo Captura de pantalla 2026-05-

☐ field name Select file

Learn more about how to set a custom [content-type](#) for a field.

Status: 200 OK Size: 41 Bytes Time: 285 ms

Response Headers ⁸ Cookies ² Results Docs </> ≡

```

1 {
2   "message": "Wallet Family en línea"
3 }

```

✖ ERROR: The file-sending feature is available only in the paid version.

5.8 Panel de Control Familiar

Método	Endpoint	Descripción	Auth
GET	/api/panel-control	Obtener datos de la familia y lista de miembros	Sí

GET http://127.0.0.1:8000/api/panel-control **Send**

Status: 200 OK Size: 960 Bytes Time: 163 ms

Query Headers **Auth 1** Body Tests Pre Run

None Basic **Bearer** OAuth 2 NTLM AWS

Bearer Token

1|zuKzWejdjcUCFZIRRK4xfaBKmYTOEN6nltfJHh

Token Prefix Bearer

Response Headers 9 Cookies Results Docs </> ≡

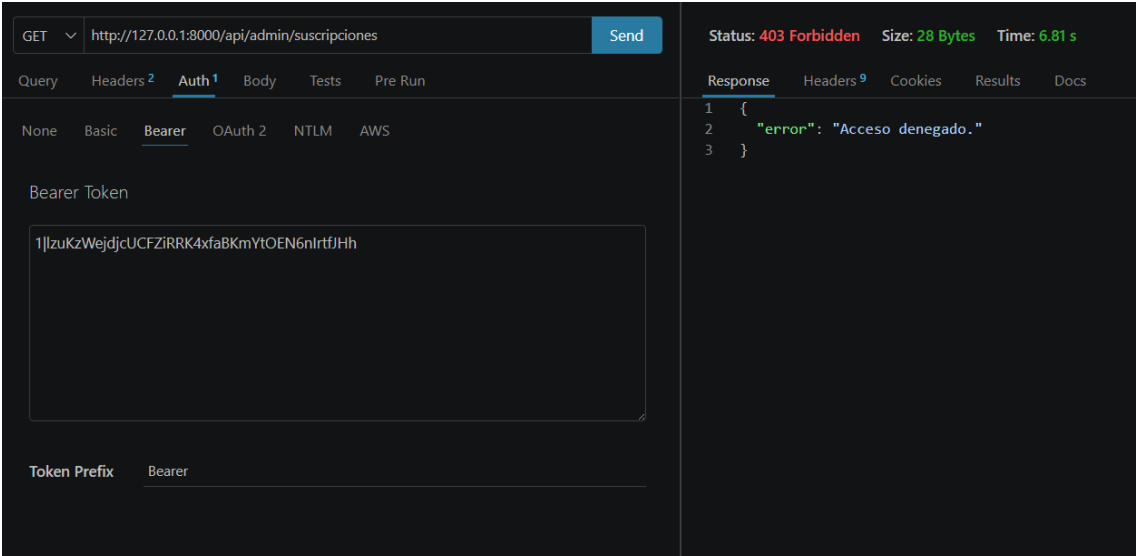
```

1 {
2   "message": "Datos de la familia y miembros recuperados exitosamente",
3   "data": {
4     "id": 12,
5     "name": "Fam lia",
6     "invitation_code": "WALLET-MN9OCL",
7     "status": "active",
8     "created_at": "2026-06-09T01:21:33.000000Z",
9     "updated_at": "2026-06-09T01:21:33.000000Z",
10    "users": [
11      {
12        "id": 12,
13        "name": "lia",
14        "email": "lia@gmail.com",
15        "email_verified_at": null,
16        "created_at": "2026-06-09T01:21:33.000000Z",
17        "updated_at": "2026-06-09T01:21:33.000000Z",
18        "last_name": "me",
19        "second_last_name": "me",
20        "birth_date": "2000-10-02T00:00:00.000000Z",
21        "phone": "12345678",
22        "profile_photo": null,
23        "family_member": "mother",
24        "status": "active",
25        "family_id": 12,
26        "role_id": 1
27      },
28      {
29        "id": 13,
30        "name": "martin",
31        "email": "martin@gmail.com",
32        "email_verified_at": null,
33        "created_at": "2026-06-09T01:21:33.000000Z",
34        "updated_at": "2026-06-09T01:21:33.000000Z",
35        "last_name": "me",
36        "second_last_name": "me",
37        "birth_date": "2000-10-02T00:00:00.000000Z",
38        "phone": "12345678",
39        "profile_photo": null,
40        "family_member": "father",
41        "status": "active",
42        "family_id": 12,
43        "role_id": 1
44      }
45    ]
46  }
47 }

```

5.9 Suscripciones

Método	Endpoint	Descripción	Auth
GET	/api/suscripcion	Ver estado actual de la suscripción y planes disponibles	Sí
POST	/api/suscripcion	Iniciar suscripción (plan: trial premium)	Sí
POST	/api/suscripcion/pagar	Subir comprobante de pago (multipart/form-data)	Sí
GET	/api/admin/suscripciones	Ver comprobantes pendientes (solo Super Admin ID=1)	Sí
POST	/api/suscripcion/admin/verificar/{family_id}	Aprobar pago y activar plan Premium	Sí
POST	/api/suscripcion/admin/rechazar/{family_id}	Rechazar comprobante de pago	Sí



5.10 Auditoría y Reportes

Método	Endpoint	Descripción	Auth
GET	/api/auditoria	Ver log de acciones. Parámetros: ?vista=personal familiar&limite=5 20 40	Sí
POST	/api/reportes/generar	Generar reporte (tipo: mensual anual top_categorias por_miembro pagos_programados)	Sí

6. CASOS DE USO

Los siguientes casos de uso describen las interacciones principales entre los actores del sistema y sus funcionalidades.

6.1 Actores del Sistema

Actor	Descripción	Rol en el sistema
Administrar de Familia	Primer usuario que crea la familia. Tiene acceso completo al panel de control.	role_id = 1
Miembro Familiar	Usuario que se une mediante código de invitación. Gestiona sus propios registros.	role_id = 2
Super Administrador	Usuario con ID=1. Verifica pagos y gestiona todas las suscripciones.	Global (sin family_id)

6.2 Descripción de Casos de Uso Principales

CU-01: Registro e Incorporación a Familia

Campo	Descripción
Actor	Usuario nuevo
Precondición	El usuario no tiene cuenta en el sistema
Flujo principal	1. El usuario accede a registro.html → 2. Elige 'Crear familia' o 'Unirme a familia' → 3. Completa datos personales (nombre, correo, contraseña, teléfono, fecha de nacimiento) → 4. Si crea: ingresa nombre de familia; si se une: ingresa código de invitación → 5. El backend crea al usuario, la familia y la suscripción Trial inicial → 6. Se retorna el token Bearer
Postcondición	Usuario autenticado con suscripción Trial activa por 30 días
Excepción	Código de invitación inválido → Error 404; Email/teléfono ya registrado → Error 422

CU-02: Registro de Ingreso

Campo	Descripción
Actor	Usuario autenticado
Precondición	Tener al menos una categoría de tipo 'income' activa
Flujo principal	1. Acceder a ingresos.html → 2. Hacer clic en 'Nuevo Ingreso' → 3. Completar descripción, monto, fecha (no futura) y categoría → 4. POST /api/ingresos → 5. El ingreso queda registrado y el saldo disponible se actualiza

Postcondición	Ingreso visible en la lista, saldo actualizado en el dashboard
Excepción	Fecha futura → Error 422 con mensaje descriptivo

CU-03: Registro de Gasto con Validación de Saldo

Campo	Descripción
Actor	Usuario autenticado
Precondición	Saldo familiar disponible $\geq$ monto del gasto
Flujo principal	1. Acceder a gastos.html → 2. Hacer clic en 'Nuevo Gasto' → 3. Completar datos → 4. POST /api/gastos → 5. El backend calcula saldo actual y verifica que el gasto no lo supere → 6. Gasto registrado
Postcondición	Gasto visible en la lista, saldo reducido
Excepción	Saldo insuficiente → Error 422: 'Tienes X Bs disponibles y estás intentando gastar Y Bs'

CU-04: Gestión de Pago Programado

Campo	Descripción
Actor	Usuario autenticado
Precondición	Categoría de tipo 'scheduled_payment' disponible
Flujo principal	1. Acceder a pagos.html → 2. Crear pago con nombre, monto y fecha límite → 3. El sistema envía SMS de recordatorio vía Twilio cuando se acerca la fecha → 4. El usuario marca el pago como 'Pagado'
Postcondición	Pago con estado 'paid' y campo paid_at registrado
Excepción	Pago vencido → Estado cambia automáticamente a 'overdue'

CU-05: Activación de Plan Premium

Campo	Descripción
Actor	Padre o Madre de familia
Precondición	La familia tiene suscripción Trial o expirada; el actor es 'father' o 'mother'
Flujo principal	1. Acceder a suscripcion.html → 2. Seleccionar plan Premium → 3. Subir comprobante de pago (imagen JPG/PNG) con referencia y monto → 4. El Super Admin recibe la solicitud en admin_suscripciones.html → 5. Admin verifica y aprueba → 6. La familia pasa a estado 'active' con plan Premium
Postcondición	Suscripción activa por 30 días adicionales
Excepción	Comprobante rechazado → Estado regresa a 'trial', la familia puede reintentar

CU-06: Generación de Reportes Financieros

Campo	Descripción
Actor	Cualquier usuario autenticado
Precondición	Existencia de datos en el período seleccionado
Flujo principal	1. Desde el Dashboard, clic en 'Generar Reporte' → 2. Seleccionar tipo (mensual, anual, top categorías, por miembro, pagos programados) → 3. POST /api/reportes/generar → 4. Se ejecuta el Stored Procedure correspondiente → 5. El reporte queda registrado en la tabla reports
Postcondición	Historial de reportes actualizado y visible en el dashboard
Excepción	Error de SP → Response 500 con detalle del error

7. MÓDULO DE ADMINISTRACIÓN

El módulo administrativo comprende las pantallas y funcionalidades accesibles al Administrador Padre/Madre de familia en sus respectivos niveles de privilegio.

7.1 Panel de Control Familiar ([panel-control.html](#))

Permite al Jefe de Familia visualizar y gestionar los miembros de su núcleo familiar.

- Ver la lista completa de integrantes con foto, nombre, rol y estado.
- Editar datos de cualquier miembro (nombre, rol familiar, contraseña).
- Expulsar miembros de la familia (DELETE /api/usuarios/{user}).
- Visualizar el código de invitación único de la familia para compartir con nuevos miembros.

7.2 Gestión de Suscripciones ([admin_suscripciones.html](#))

Exclusivo para el Super Administrador (ID=1). Permite gestionar el ciclo de pagos de todas las familias.

- Ver la tabla de familias con comprobantes pendientes de verificación.
- Visualizar la imagen del comprobante de pago subida por la familia.
- Aprobar un pago: activa el plan Premium por 30 días (POST /api/suscripcion/admin/verificar/{family_id}).
- Rechazar un pago: elimina el comprobante y devuelve a la familia al estado Trial para que pueda reintentar (POST /api/suscripcion/admin/rechazar/{family_id}).

7.3 Gestión de Categorías ([categorias.html](#))

Accesible a todos los usuarios. Cada familia administra sus propias categorías de forma aislada.

- Crear categorías para clasificar ingresos, gastos o pagos programados.
- Editar el nombre y descripción de una categoría existente.
- Activar o desactivar categorías sin eliminarlas (toggle de estado).

7.4 Auditoría de Seguridad ([auditoria.html](#))

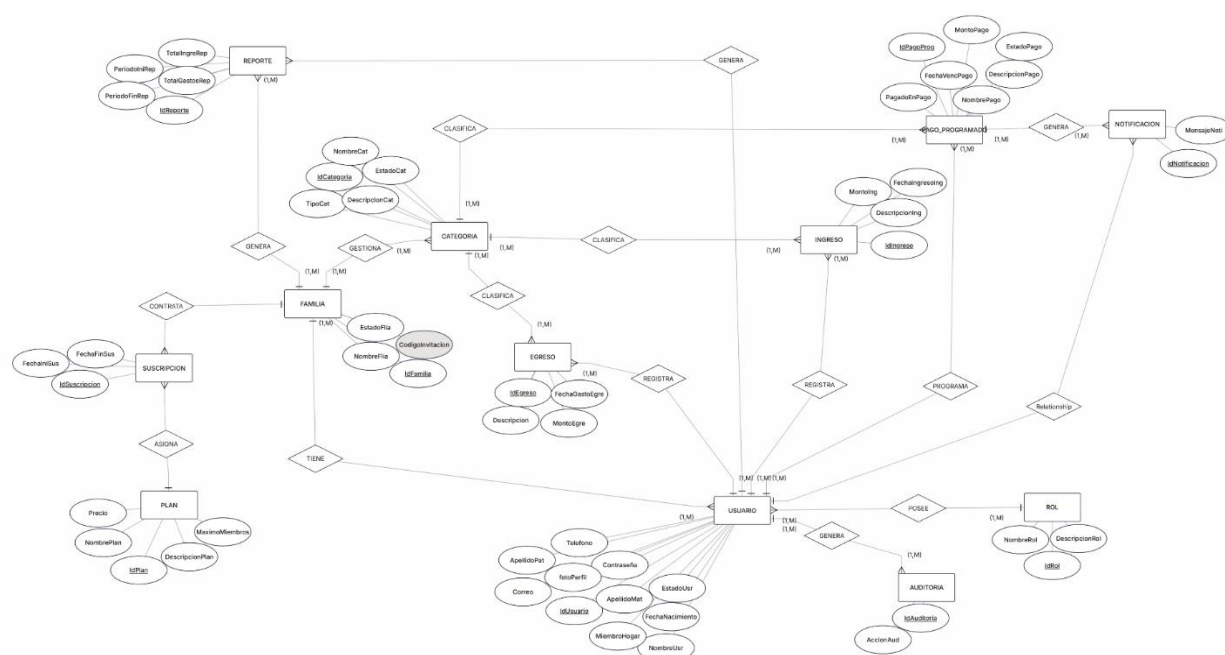
Registro de acciones del sistema almacenado vía Stored Procedures.

- Vista familiar: muestra las acciones de todos los miembros de la familia.
- Vista personal: muestra solo las acciones del usuario autenticado.
- Control de carga: límite configurable de 5, 20 o 40 registros.

8. MODELO ENTIDAD RELACIÓN

El modelo de datos está compuesto por 12 entidades que reflejan la lógica de negocio del sistema financiero familiar. A continuación se describen las relaciones entre ellas:

8.1 Relaciones Principales



Entidad Padre	Relación	Entidad Hija	Cardinalidad	Campo de enlace
families	tiene	users	1:N	users.family_id
families	tiene	categories	1:N	categories.family_id
families	tiene	subscriptions	1:N	subscriptions.family_id
families	genera	reports	1:N	reports.family_id
roles	asigna	users	1:N	users.role_id
users	registra	incomes	1:N	incomes.user_id
users	registra	expenses	1:N	expenses.user_id
users	programa	scheduled_payments	1:N	scheduled_payments.user_id
users	genera	reports	1:N	reports.user_id
users	genera	audits	1:N	audits.user_id
categories	clasifica	incomes	1:N	incomes.category_id
categories	clasifica	expenses	1:N	expenses.category_id
categories	clasifica	scheduled_payments	1:N	scheduled_payments.category_id
scheduled_payments	genera	notifications	1:N	notifications.scheduled_payment_id
plans	define	subscriptions	1:N	subscriptions.plan_id

□ El aislamiento de datos por familia se garantiza en el código: cada controlador filtra por family_id del usuario autenticado antes de ejecutar cualquier operación CRUD. Esta es una capa de seguridad a nivel de aplicación complementaria a las llaves foráneas de la base de datos.

9. DICCIONARIO DE DATOS

9.1 Tabla: users

Campo	Tipo	Nulo	Descripción
id	BIGINT UNSIGNED	NO	Identificador único autoincremental
name	VARCHAR(255)	NO	Nombre(s) del usuario
last_name	VARCHAR(80)	NO	Primer apellido
second_last_name	VARCHAR(80)	SÍ	Segundo apellido (opcional)
email	VARCHAR(255)	NO	Correo electrónico (único en el sistema)
password	VARCHAR(255)	NO	Contraseña hasheada con bcrypt
birth_date	DATE	NO	Fecha de nacimiento (mínimo 15 años)
phone	VARCHAR(20)	NO	Teléfono de 8 dígitos (único)
profile_photo	VARCHAR(255)	SÍ	Ruta relativa de la foto de perfil
family_member	ENUM	NO	Rol en la familia: father, mother, child
status	ENUM	NO	Estado de la cuenta: active, inactive
family_id	BIGINT UNSIGNED	SÍ	FK → families.id
role_id	BIGINT UNSIGNED	SÍ	FK → roles.id (1=Jefe, 2=Miembro)

9.2 Tabla: families

Campo	Tipo	Nulo	Descripción
id	BIGINT UNSIGNED	NO	Identificador único
name	VARCHAR(255)	NO	Nombre elegido para la familia
invitation_code	VARCHAR(20)	NO	Código único para invitar miembros (ej: WALLET-ABC123)
status	ENUM	NO	Estado: active, inactive

9.3 Tabla: incomes (Ingresos)

Campo	Tipo	Nulo	Descripción
id	BIGINT UNSIGNED	NO	Identificador único
description	VARCHAR(255)	NO	Descripción del ingreso
amount	DECIMAL(10,2)	NO	Monto en Bolivianos (mínimo 0.01)
income_date	DATE	NO	Fecha del ingreso (no puede ser futura)
user_id	BIGINT UNSIGNED	NO	FK → users.id
category_id	BIGINT UNSIGNED	NO	FK → categories.id (tipo: income)

9.4 Tabla: expenses (Gastos)

Campo	Tipo	Nulo	Descripción
id	BIGINT UNSIGNED	NO	Identificador único
description	VARCHAR(255)	NO	Descripción del gasto
amount	DECIMAL(10,2)	NO	Monto (validado contra saldo disponible)
expense_date	DATE	NO	Fecha del gasto
user_id	BIGINT UNSIGNED	NO	FK → users.id
category_id	BIGINT UNSIGNED	NO	FK → categories.id (tipo: expense)

9.5 Tabla: scheduled_payments (Pagos Programados)

Campo	Tipo	Nulo	Descripción
id	BIGINT UNSIGNED	NO	Identificador único
name	VARCHAR(255)	NO	Nombre del pago (ej: Alquiler, Netflix)
amount	DECIMAL(10,2)	NO	Monto del pago
deadline_date	DATE	NO	Fecha límite de pago
status	ENUM	NO	Estado: pending, paid, overdue
paid_at	DATETIME	SÍ	Fecha/hora en que se marcó como pagado
user_id	BIGINT UNSIGNED	NO	FK → users.id
category_id	BIGINT UNSIGNED	NO	FK → categories.id (tipo: scheduled_payment)

9.6 Tabla: categories

Campo	Tipo	Nulo	Descripción
id	BIGINT UNSIGNED	NO	Identificador único
name	VARCHAR(255)	NO	Nombre de la categoría
type	ENUM(income, expense, scheduled_payment)	NO	Tipo de transacción que clasifica
description	TEXT	SÍ	Descripción opcional
status	ENUM(active, inactive)	NO	Estado de la categoría
family_id	BIGINT UNSIGNED	NO	FK → families.id (aislamiento de familia)

9.7 Tabla: subscriptions

Campo	Tipo	Nulo	Descripción
id	BIGINT UNSIGNED	NO	Identificador único
family_id	BIGINT UNSIGNED	NO	FK → families.id
plan_id	BIGINT UNSIGNED	NO	FK → plans.id
status	ENUM	NO	trial pending_payment active expired
payment_status	ENUM	NO	free pending verified rejected
start_date	DATE	NO	Inicio de la suscripción
end_date	DATE	NO	Fin de la suscripción
payment_reference	VARCHAR(100)	SÍ	Número de referencia bancaria
payment_amount	DECIMAL(10,2)	SÍ	Monto pagado según el comprobante
payment_date	DATE	SÍ	Fecha del pago reportado
image_path	VARCHAR(255)	SÍ	Ruta del comprobante de pago
payment_verified_at	DATETIME	SÍ	Fecha de verificación por el Admin
verified_by	BIGINT UNSIGNED	SÍ	FK → users.id del administrador

9.8 Tabla: plans

Campo	Tipo	Nulo	Descripción
id	BIGINT UNSIGNED	NO	Identificador único
name	VARCHAR(100)	NO	Nombre del plan (Trial, Premium)
price	DECIMAL(8,2)	NO	Precio mensual en Bolivianos
max_members	INT	NO	Número máximo de miembros permitidos
description	TEXT	SÍ	Descripción de beneficios del plan

10. PROTOTIPOS DE PANTALLAS DEL APLICATIVO

El frontend de Wallet Family está compuesto por 14 páginas HTML que consumen la API REST. Todas utilizan Bootstrap 5 para diseño responsivo y un sidebar de navegación lateral en escritorio con menú deslizante (offcanvas) en dispositivos móviles.

10.1 Inventario de Pantallas

Archivo	Pantalla	Acceso	Descripción
index.html	Inicio / Bienvenida	Público	Página de presentación con acceso a login y registro
login.html	Iniciar Sesión	Público	Formulario de login con detección de parámetro ?redirect=suscripcion
registro.html	Registro de Usuario	Público	Formulario unificado para crear familia o unirse con código
dashboard.html	Resumen Familiar	Privado	Cards de saldo, últimos movimientos, historial de reportes
ingresos.html	Gestión de Ingresos	Privado	CRUD de ingresos con filtro por mes y categoría
gastos.html	Gestión de Gastos	Privado	CRUD de gastos con validación de saldo disponible
categorias.html	Gestión de Categorías	Privado	CRUD de categorías con toggle de activación
pagos.html	Pagos Programados	Privado	CRUD de pagos con filtros por mes y estado
perfil.html	Mi Perfil	Privado	Edición de datos personales y foto de perfil
panel-control.html	Panel de Control	Privado	Gestión de miembros familiares (solo Jefe/Madre)
suscripcion.html	Suscripción	Privado	Ver plan actual, elegir plan, subir comprobante de pago
admin_suscripciones.html	Admin Suscripciones	Super Admin	Verificar/rechazar comprobantes de familias
auditoria.html	Auditoría de Seguridad	Privado	Log de acciones personal o familiar

10.2 Estructura de Navegación Común

Todas las páginas privadas comparten la siguiente estructura de layout:

- **Sidebar desktop:** Menú lateral fijo con Bootstrap nav-pills. Incluye accesos a todas las secciones del sistema.
- **Navbar móvil:** Barra superior con botón hamburguesa que despliega un offcanvas.
- **Área de contenido principal:** Columna central con cards, tablas y modals de Bootstrap 5.
- **Sesión:** El nombre del usuario logueado se carga dinámicamente desde localStorage al iniciar cada página.

10.3 Descripción Detallada de Pantallas Clave

Dashboard (dashboard.html)

Pantalla central del sistema. Muestra:

- Tres cards de resumen: Total Ingresos, Total Gastos y Saldo Disponible (calculado como ingresos - gastos).
- Selector de vista: Familiar (datos de toda la familia) o Personal (solo el usuario).
- Tabla de últimos movimientos con diferenciación visual entre ingresos (verde) y gastos (rojo).
- Sección de reportes: selector de tipo y botón para generación; historial de reportes generados.
- Control de límite: selector de 5, 20 o 40 registros por vista.

Registro (registro.html)

Formulario multistep que adapta sus campos según la acción elegida:

- **Crear familia:** Muestra campo de nombre de familia. Al registrar se genera el código de invitación automáticamente.
- **Unirme a familia:** Muestra campo de código de invitación. El backend valida que el código exista.
- Validación de edad mínima de 15 años (before_or_equal:-15 years).
- Campo de rol familiar: Padre, Madre o Hijo/a.

Suscripción (suscripcion.html)

Flujo completo de gestión de suscripción en una sola página:

- Estado 1 – Trial: Muestra días restantes del período de prueba y botón para actualizar a Premium.
- Estado 2 – Pendiente de pago: Formulario para subir comprobante (referencia, monto, fecha, imagen del comprobante).
- Estado 3 – Activo: Muestra fecha de expiración del plan Premium.

- Estado 4 – Rechazado: Aviso de rechazo con opción de volver a subir comprobante.

11. ARQUITECTURA DEL SISTEMA Y SEGURIDAD

11.1 Arquitectura General

Wallet Family sigue una arquitectura Cliente-Servidor desacoplada:

Capa	Tecnología	Responsabilidad
Presentación (Frontend)	HTML5 + Bootstrap 5 + JS	Interfaz de usuario, consumo de la API, manejo del token en localStorage
Lógica de Negocio (Backend)	Laravel 9 (PHP 8)	Controladores, validaciones, políticas de acceso, integración Twilio
Persistencia (Base de Datos)	MySQL 8.0	Almacenamiento, relaciones, Stored Procedures para reportes y auditoría

11.2 Mecanismos de Seguridad Implementados

- **Autenticación con Laravel Sanctum:** Cada solicitud a endpoints protegidos requiere el encabezado Authorization: Bearer {token}. El token se revoca en logout.
- **Aislamiento de familia:** Todos los controladores verifican que el recurso solicitado pertenezca a la misma family_id del usuario autenticado antes de retornarlo o modificarlo.
- **Autorización por Policy:** ExpensePolicy, IncomePolicy y ScheduledPaymentPolicy verifican que solo el propietario (o miembro de la misma familia) pueda editar/eliminar un registro.
- **Validación de saldo negativo:** ExpenseController calcula el saldo disponible antes de registrar un gasto y rechaza la operación si el monto supera el saldo.
- **Restricción de edad mínima:** El registro valida que el usuario tenga al menos 15 años mediante la regla before_or_equal:-15 years.
- **Protección de foto de perfil:** ProfilePhotoController verifica que Auth::id() === user->id antes de permitir la modificación.
- **Restricciones del Super Admin:** Las rutas administrativas verifican Auth::id() === 1 directamente en el controlador.
- **Transacciones de BD:** El registro de usuario usa DB::beginTransaction() con rollback automático ante cualquier fallo.
- **CORS configurado:** El paquete fruitcake/laravel-cors gestiona los encabezados de origen cruzado de forma centralizada.